

vision Day1 hw2 AI 보고서

로봇 21기 예비단원 최윤서

1. 왜 사용했는가?

코드를 모두 작성한 후 작동시켜봤을 때 빨간색 부분이 여러 객체로 검출됨
교육시간에 배웠던 Hue값 때문이라는 것을 인지했지만 그 후로 어떻게 처리를 해야할지
감이 잡히지않았고, 침식, 팽창, 열기, 닫기 중에 어떤걸 사용해야할지 모르겠어서 AI에게
질문함.

2. 질문 사진

```
#include
"color_object_detection/color_object_detection.hpp"

ColorObjectDetection::ColorObjectDetection()
: Node("color_object_detection")
{
    RCLCPP_INFO(this->get_logger(), "Color Object
Detection Node Started");

    this->declare_parameter<std::string>("image_path", "");
}

Show more ▾
```

현재 코드에서 빨간색 물체 하나가 여러 개의 객체로 검출되는 문제라면, 빨간색 mask가 하나의 연
결된 영역으로 만들어지지 않아 findContours() 에서 여러 contour가 생성되는 경우를 먼저 처
리하는 게 좋습니다.

수정할 부분

red_mask = red_mask_1 | red_mask_2; 바로 다음에 다음 코드를 넣으세요.

```
// C++

cv::Mat kernel = cv::getStructuringElement(
    cv::MORPH_ELLIPSE,
    cv::Size(15, 15)
);

cv::morphologyEx(
    red_mask,
    red_mask,
    cv::MORPH_CLOSE,
    kernel
);
```

즉, 빨간색 부분 전체가 다음과 같이 됩니다.

즉, 빨간색 부분 전체가 다음과 같이 됩니다.

```
// C++

cv::inRange(
    hsv_image,
    cv::Scalar(0, 70, 50),
    cv::Scalar(10, 255, 255),
    red_mask_1
);

cv::inRange(
    hsv_image,
    cv::Scalar(170, 70, 50),
    cv::Scalar(179, 255, 255),
    red_mask_2
);

// 두 개의 빨간색 mask를 하나로 합침
red_mask = red_mask_1 | red_mask_2;

// 빨간색 영역의 끊어진 부분을 연결
cv::Mat kernel = cv::getStructuringElement(
    cv::MORPH_ELLIPSE,
    cv::Size(15, 15)
);

cv::morphologyEx(
    red_mask,
    red_mask,
    cv::MORPH_CLOSE,
    kernel
);
```

왜 이렇게 하는가?

현재 객체 검출은 결국 다음 과정입니다.

```
red_mask
|
findContours()
|
여러 contour 생성
|
각 contour를 하나의 객체로 판단
```

따라서 빨간색 물체 하나의 mask가 여러 영역으로 끊어져 있으면 물체 하나인데도 여러 contour가
생성될 수 있습니다.

MORPH_CLOSE를 적용하면 작은 틈이나 구멍을 메우고 가까운 영역을 연결하여 하나의 contour로
만들 가능성이 높아집니다.

그리고 현재 코드의

```
// C++
```

도 그대로 유지하면 작은 잡음 영역은 추가로 제거할 수 있습니다.

따라서 현재 코드에서는 findContours() 부분을 수정하는 것보다, 빨간색 mask를 만드는 단계
에서 Closing을 적용하는 방식으로 수정하는 것이 적절합니다.

3. 느낀점

교육시간에도 빨간색 부분이 왜 두개로 나뉘어져야하는지 제대로 이해하지 못해
선배님께 질문했었습니다. 질의응답을 통해 개념을 이해할 수 있었지만 코드로 구현하는
것은 개념 이해와는 별개라는 것을 느꼈습니다. 개념을 이해했어도 어떤 함수를 사용하고
어떤 변수를 넣어야하는지, 코드에 어느 부분에 넣어야하는지는 개념 이해만으로는 해결할
수 없었습니다.

앞으로는 개념 이해에서 그치지않고 코드 구현까지 간단한 예제를 통해 연습해보는 습관을
들여야겠다고 느꼈습니다.